Project Title

Student record management system using array

Submitted to : Ms. Konika Grover

Subject Name: Data structure

Subject Code : 25 CAP-152

Submitted by : Jashandeep kaur

UID : 25BCA10034

Student Record Management System Using Array

Introduction

A Student Record Management System is a software application used to store, manage, and retrieve student information efficiently. It helps in maintaining records such as student name, roll number, and marks in an organized way.

In this project, we use the **array data structure** to store multiple student records. An array is a linear data structure that stores elements of the same type in contiguous memory locations.

This project is very useful for beginners as it helps in understanding basic programming concepts like data storage, structures, and functions. It also demonstrates how data can be handled in a simple and efficient way.

Objectives

The main objectives of this project are:

- To manage student records efficiently
- To understand the concept of arrays
- To perform operations like add, display, and search
- To improve programming and logical skills
- To implement a real-life application using C programming

Features

The Student Record Management System provides useful features that help in managing student records easily and efficiently.

1. Add New Student Records

The system allows the user to add student details such as roll number, name, and marks. These records are stored in the array.

2. Display All Records

The user can view all stored student records together. It shows roll number, name, and marks of each student.

3. Search Student by Roll Number

The system can search a student record using the roll number. This makes finding a specific record easy and fast.

4. User-Friendly Interface

The program uses a simple menu-driven interface. Users can easily choose options like Add, Display, Search, or Exit.

5. Fast Data Access

Since arrays are used, records can be accessed quickly and efficiently.

6. Time Saving

The system saves time by managing records in a digital and organized way.

Data Structure

This project uses an **array** to store student records.

Each student is represented using a structure that contains:

- Roll Number
- Name
- Marks

Example:

```
Strict Student {
   into roll;
   char name [50];
   float marks;
};
```

The array is declared as:

```
Strict Student s[100];
```

This means we can store records of up to 100 students.

Algorithm

1. Start
2. Define the student structure
3. Declare an array to store student records
4. Display menu options
5. Take input from the user
6. Perform the selected operation (add/display/search)
7. Repeat the process until the user exits
8. Stop

Source Code

```
#include <stdio.h>

#include <string.h>


struct Student {

   int roll;

   char name[50];

   float marks;

};


struct Student s[100];

int count = 0;


void addStudent() {

   printf("Enter Roll No: ");

   scanf("%d", &s[count].roll);


   printf("Enter Name: ");

   scanf("%s", s[count].name);


   printf("Enter Marks: ");

   scanf("%f", &s[count].marks);


   count++;

}
```

```
void displayStudent() {
    for(int i = 0; i < count; i++) {
        printf("%d %s %.2f\n", s[i].roll, s[i].name, s[i].marks);
    }
}

void searchStudent() {
    int roll, found = 0;

    printf("Enter Roll No to search: ");
    scanf("%d", &roll);

    for(int i = 0; i < count; i++) {
        if(s[i].roll == roll) {
            printf("Student Found: %s %.2f\n", s[i].name, s[i].marks);
            found = 1;
        }
    }

    if(found == 0)
        printf("Student not found\n");
}

int main() {
```

```
    int choice;

    while(1) {
        printf("\n1.Add 2.Display 3.Search 4.Exit\n");
        scanf("%d", &choice);

        switch(choice) {
            case 1: addStudent(); break;
            case 2: displayStudent(); break;
            case 3: searchStudent(); break;
            case 4: return 0;
        }
    }
}
```

Sample Output

1.Add 2.Display 3.Search 4.Exit

Enter choice: 1

Enter Roll No: 1

Enter Name: Jashan

Enter Marks: 90

1 jashan 90.00

Advantages

1. Easy to Understand and Implement

The project is simple in design and coding. Beginners can easily understand how the system works.

2. Fast Data Access

Since arrays use indexes, student records can be accessed quickly.

3. Suitable for Beginners

This project is very helpful for students who are learning programming concepts.

4. Requires Less Memory

Arrays store data in a simple way, so the system uses less memory for small records.

5. Simple Logic

The program uses basic operations like add, display, and search, which are easy to learn.

Limitations

1. Fixed Size of Array

The array size is fixed, so only a limited number of student records can be stored.

2. Not Suitable for Large Data

If the number of students is very high, arrays are not the best option.

3. Insertion and Deletion are Difficult

Adding or deleting records in between the array can take more time.

4. Cannot Use Dynamic Memory

Arrays cannot increase or decrease in size during program execution.

Conclusion

The Student Record Management System is a simple yet effective project that helps in understanding the use of arrays in managing data. It demonstrates how student records can be stored, retrieved, and manipulated efficiently.

In the future, this system can be enhanced by using advanced data structures like linked lists or databases and adding a graphical user interface (GUI) for better user experience.